

CHAPTER 19

Direct Memory Access (DMA)

Direct memory access (DMA) is a useful technique for transferring data between peripherals and memory, or between memory and memory, without using many processor cycles. The processor needs to program the DMA controller and send a command to start the DMA transfer. However, during the transfer process, the processor is not involved and can execute other tasks. Therefore, DMA is a very efficient and widely-used approach to interface peripherals.

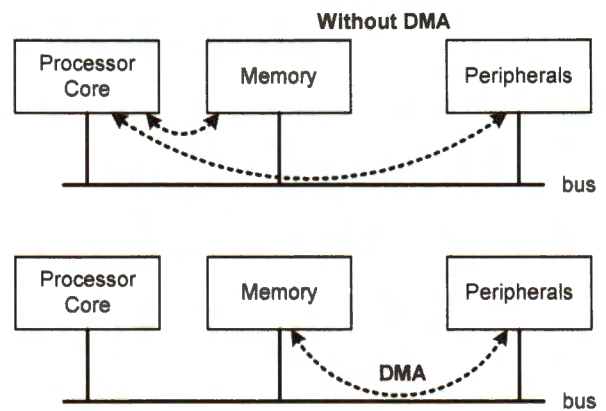


Figure 19-1. DMA runs on behalf of the processor

- For slow peripherals, DMA releases the processor from waiting for peripheral data and allows the CPU to serve other tasks.
- For fast peripherals, such as ADC and DAC, DMA improves the data transfer throughput because memory accesses take place without the involvement of the processor.
- For high-speed peripherals, DMA can help significantly reduce the rate at which interrupts are generated. Thus, DMA decreases the performance overhead from interrupts.

With DMA support, a peripheral does not need to have local memory to store data. Instead, data can be efficiently stored in data memory via DMA. This design not only reduces the cost but also improves the energy efficiency of embedded systems. This chapter presents the implementation of the bus matrix within the processor and gives an example of how to program a DMA controller.

19.1 Advanced Microcontroller Bus Architecture (AMBA)

The Advanced Microcontroller Bus Architecture (AMBA) is an open on-chip communication standard for embedded microcontrollers. Chip designers can use AMBA without paying royalties. AMBA was first introduced by ARM in 1996. AMBA 2.0, released in 1999, specifies the architecture and communication protocol of three bus standards, including the Advanced High-performance Bus (AHB), the Advanced System Bus (ASB), and the Advanced Peripheral Bus (APB). A bus is a set of physical connections that allows two or more components to communicate. Each bus architecture provides a set of separate communication paths to transfer data, instructions, and control signals. AMBA deploys a master/slave communication model in which only a master can control the bus and initiate data transfer.

- The AHB or ASB is the backbone bus that has high-performance and high clock frequency. It supports pipelined operation to achieve high bandwidth, burst transfers to reduce the latency of bus access arbitration, and multiple bus masters to allow several components to initiate data transfer. Bus arbitration refers to the process of deciding which bus master will be allowed to control the bus when there are multiple masters. AHB supports wider data transfer and has a higher bandwidth than ASB. AHB is used on ARM Cortex-M microprocessors.
- The APB is a simple, low-power, and low-bandwidth bus that is suitable for low-speed peripherals such as timers, USART, SPI, and LCD. These peripherals typically use memory-mapped registers to provide a simplified interface. Unlike AHB, communications in APB are not pipelined. All peripherals on the APB bus can only act in the role of a slave, which cannot initiate a transfer.

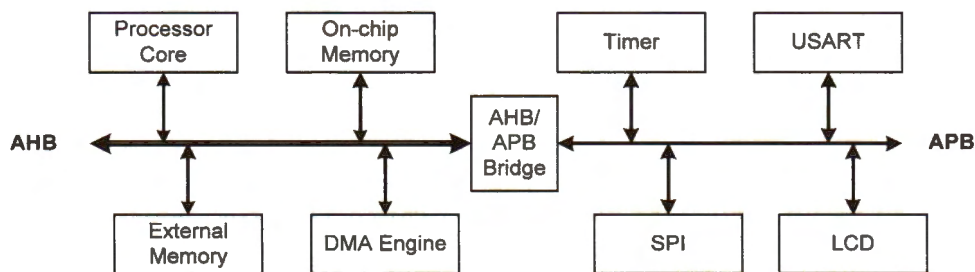


Figure 19-2. A simple system consisting of AHB, APB, and AHB/APB bridge

The AHB bus is connected to a bus matrix. The matrix uses a priority-based round-robin scheduling algorithm to ensure that no modules can block the data transmission of the other modules for a long time. The bus matrix also arbitrates any conflicts when multiple accesses are attempted simultaneously.

A bridge is required to connect the AHB and APB buses. The bridge allows a master on the AHB bus to communicate with a peripheral device on the APB bus, even though they use different communication protocols. The bridge acts as the only master on the APB bus and as a slave on the AHB bus. Due to the speed gap between AHB and APB, the bridge buffers addresses, controls and data from AHB to ensure that there is no data loss during the communications. The bridge also drives the APB peripherals and provides response signals to the AHB.

DMA controllers reside on the AHB bus. A DMA controller acts as a bus master and a bus slave. It has two ports. The slave port can accept data and commands from the processor when the processor sets up DMA transfers. The master port can initiate data transfer within the AHB bus or across the AHB/APB bridge. A DMA controller often manages multiple channels that can be programmed independently. While all channels of a DMA controller share the same interface to the AHB bus, each channel has its own dedicated interface to peripherals. Thus, multiple channels can perform DMA transfers simultaneously.

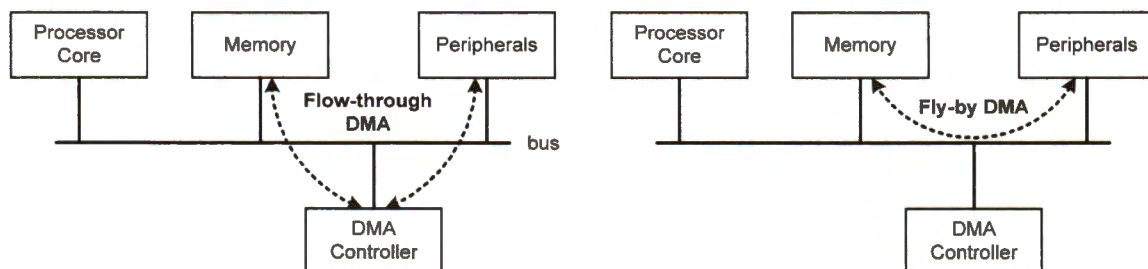


Figure 19-3. Flow-through *vs* fly-by DMA

DMA transfers can be either *flow-through* or *fly-by*, as shown in Figure 19-3.

- A *flow-through* (or fetch-and-deposit) DMA transfer involves two steps: (1) data are read one at a time from the source into a register in the DMA controller, and (2) data are written to the destination.
- A *fly-by* DMA transfer only requires one bus transfer. Data are directly transferred from the source to the destination without being read into the DMA controller.

Flow-through transfers are less efficient than fly-by transfers. However, flow-through transfers can be used for DMA operations between devices with different register sizes. For example, it can read twice from a 16-bit source and then write once to a 32-bit destination. Also, fly-by transfers cannot be used for memory-to-memory DMA in any embedded systems in which memory cannot be read and written in the same memory clock cycle. The DMA controllers on STM32L processors use flow-through transfers.

19.2 Interfacing a Peripheral without and with DMA

Figure 19-4 and Figure 19-5 compare operations when data is transferred from the serial port (USART) to RAM without and with DMA. STM32L4 divides on-chip peripherals into two groups, with one APB bus for each group (APB1 and APB2). There are two AHB-to-APB bridges that connect the AHB bus matrix to APB1 and APB2, respectively. STM32L4 provides two DMA controllers, offering 7 channels on each controller. Only one DMA controller is shown in Figure 19-4 and Figure 19-5.

Without DMA, the processor is fully occupied during the data transfer. It needs to perform two operations explicitly to read data from a peripheral. The first operation is a load instruction that copies a peripheral data register to a register in the processor core. The second operation is a store instruction that saves data to memory.

Similarly, the processor must perform a load and a store operation to write data to a peripheral. The processor reads the data from memory and writes the data to the peripheral.

Furthermore, without DMA, the processor uses either busy-waiting or interrupt methods to make sure that the peripheral's data register is ready to read or write. The busy-waiting method is simple but inefficient because it prevents the processor from performing other tasks. The interrupt approach is more efficient than busy-waiting. However, it is not suitable for high-speed peripherals because high-frequency interrupts can create a large performance overhead. Chapter 22.1.4 and Chapter 22.1.5 give a detailed implementation of sending and receiving data from a USART serial port by using busy-waiting (also known as polling) and interrupts.

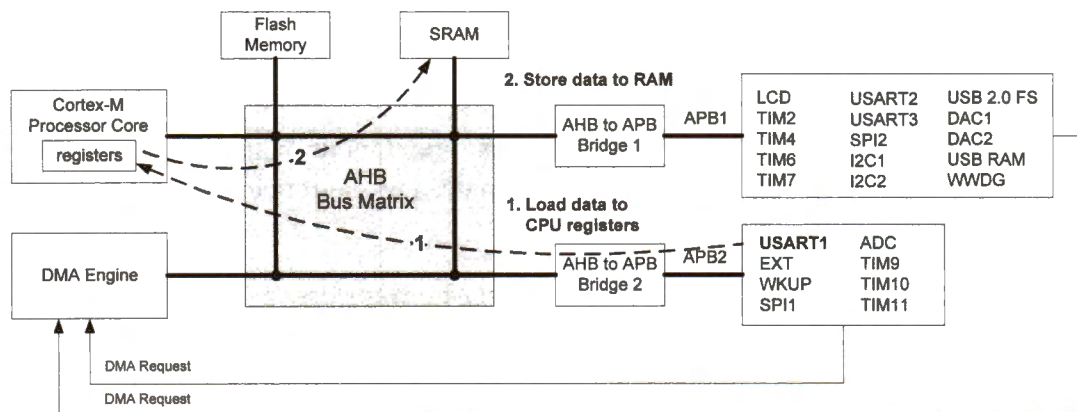


Figure 19-4. Receiving data from a USART serial port without DMA. The processor must execute load and store instructions to copy data from peripherals to the data memory.

On the contrary, if DMA is used, data exchange between memory and peripherals, or between two different memory regions, occurs without processor intervention. The processor only needs to set up the DMA transfers by programming the DMA controller and start the transfer by enabling the DMA channel. Once the transfer is in progress, the processor becomes available for other tasks.

For example, after the processor configures and initiates the DMA channel that responds to the USART1 *receive register not empty* (RXNE) events, the DMA engine copies data in the USART1 RX register to memory as soon as it is received.

- The “write to” and “read from” operations are offloaded from the processor core to the DMA controller.
- The DMA engine uses flow-through transfers. Each DMA transfer still takes two bus transfers to move data from USART1 to memory. Although DMA does not reduce the total latency of these two data transfers, this latency can be hidden from the processor.
- The DMA controller can generate an interrupt at the end of each DMA transfer sequence to inform the processor that data has been saved to memory and is ready to be processed.

In this way, DMA allows data computation and data transfer to take place in parallel, leading to performance improvement. DMA may stop the processor’s access to the AHB bus for a short time if both are trying to access the same destination (memory or peripherals) at the same time.

Chapter 22.1.6 provides a detailed example of controlling USART 1 via DMA.

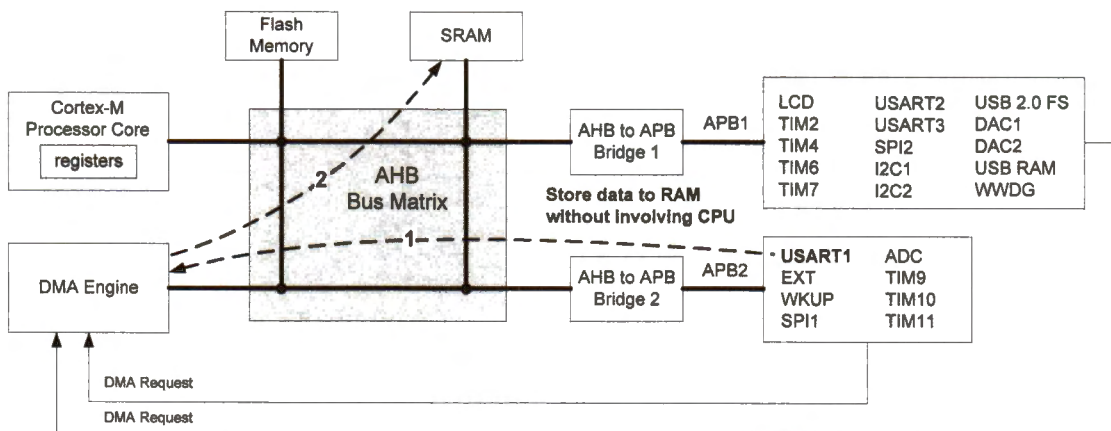


Figure 19-5. Receiving data from USART 1 by using DMA. The DMA engine reads data from the peripheral and then writes it to memory without the intervention of the processor. The DMA engine can inform the processor via interrupt upon completion.

19.3 DMA Channels

Let us take the STM32L4 as an example to illustrate the concept of a DMA channel. STM32L4 has two DMA controllers, with seven channels in each. Each channel can perform DMA transfers independently. Each channel has its own source, destination, transfer direction, transfer width, data amount, and trigger.

If a DMA channel is enabled, it performs a DMA transfer automatically for each trigger received. Figure 19-6 shows one DMA controller. Each channel can be programmed to be triggered by one event selected among eight events available to that channel. This is achieved by programming DMA controller's CSELR register. Software trigger is one of the eight trigger options on every channel.

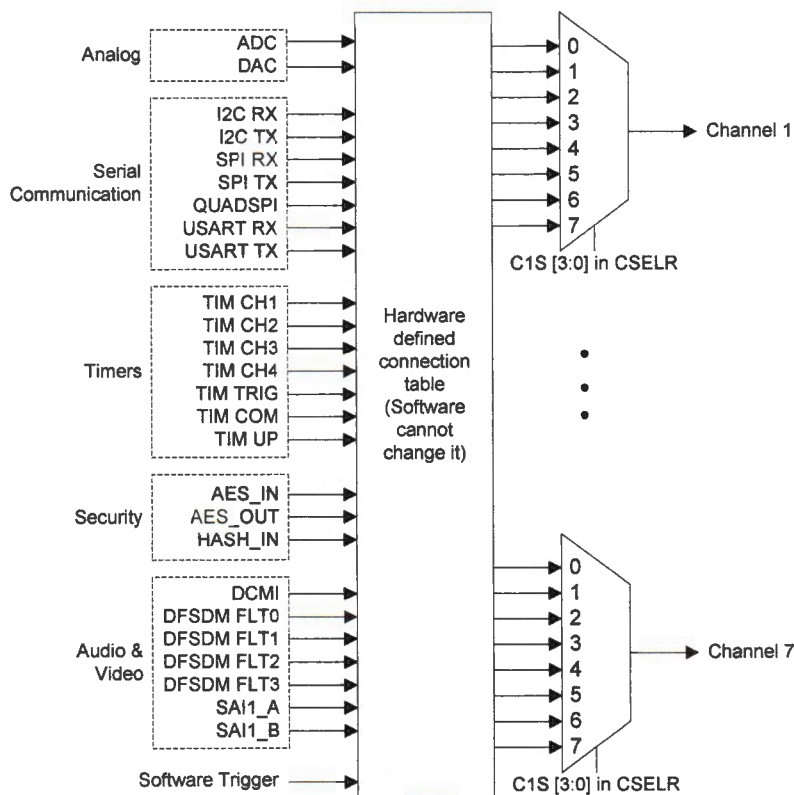


Figure 19-6. Selecting one trigger signal for each DMA channel

To reduce a chip's cost and complexity, each channel can only support up to eight trigger signals (also called requests) defined by the chip manufacturer. For a given DMA channel, software can only select one available trigger for that channel. In other words, not all triggers are available for a channel at the same time.

Table 19-1 and Table 19-2 show the possible triggers on each channel of DMA 1 and DMA 2. To avoid DMA conflicts, we need to consider the following questions when programming DMA: (1) Which DMA controller should we use? (2) Which channel should we use? (3) Which trigger should we select?

Trigger	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Channel 6	Channel 7
0	ADC1	ADC2	ADC3	DFSDM1_FLT0	DFSDM1_FLT1	DFSDM1_FLT2	DFSDM1_FLT3
1	-	SPI1_RX	SPI1_TX	SPI2_RX	SPI2_TX	SAI2_A	SAI2_B
2	-	USART3_TX	USART3_RX	USART1_TX	USART1_RX	USART2_RX	USART2_TX
3	-	I2C3_TX	I2C3_RX	I2C2_TX	I2C2_RX	I2C1_TX	I2C1_RX
4	TIM2_CH3	TIM2_UP	TIM16_CH1 TIM16_UP	-	TIM2_CH1	TIM16_CH1 TIM16_UP	TIM2_CH2 TIM2_CH4
5	TIM17_CH1 TIM17_UP	TIM3_CH3	TIM3_CH4 TIM3_UP	TIM7_UP DAC2	QUADSPI	TIM3_CH1 TIM3_TRIG	TIM17_CH1 TIM17_UP
6	TIM4_CH1	-	TIM6_UP DAC1	TIM4_CH2	TIM4_CH3	-	TIM4_UP
7	-	TIM1_CH1	TIM1_CH2	TIM1_CH4 TIM1_TRIG TIM1_COM	TIM15_CH1 TIM15_UP TIM15_TRIG TIM15_COM	TIM1_UP	TIM1_CH3

Table 19-1. DMA triggers (requests) supported in each channel of DMA controller 1

Trigger	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Channel 6	Channel 7
0	I2C4_RX	I2C4_TX	ADC1	ADC2	ADC3	DCMI	-
1	SAI1_A	SAI1_B	SAI2_A	SAI2_B	-	SAI1_A	SAI1_B
2	UART5_TX	UART5_RX	UART4_TX	-	UART4_RX	USART1_TX	USART1_RX
3	SPI3_RX	SPI3_TX	-	TIM6_UP DAC1	TIM7_UP DAC2		QUADSPI
4	SWPMI1_RX	SWPMI1_TX	SPI1_RX	SPI1_TX	DCMI	LPUART1_TX	LPUART1_RX
5	TIM5_CH4 TIM5_TRIG	TIM5_CH3 TIM5_UP	-	TIM5_CH2	TIM5_CH1	I2C1_RX	I2C1_TX
6	AES_IN	AES_OUT	AES_OUT	-	AES_IN	-	HASH_IN
7	TIM8_CH3 TIM8_UP	TIM8_CH4 TIM8_TRIG TIM8_COM	-	SDMMC1	SDMMC1	TIM8_CH1	TIM8_CH2

Table 19-2. DMA triggers (requests) supported in each channel of DMA controller 2

Each channel has a software priority and a hardware priority. When multiple channels are active, the DMA controller uses the priority level to determine the order of bus access when a bus resource conflict occurs. If two or more DMA channels have the same software priority level, the DMA controller uses the hardware priority to break the tie.

- **Software priority.** Programs can configure the software priority of each channel. When multiple channels are used, software should give a high priority to a channel that requires a high bandwidth, to achieve continuous data transfers and provide sustained data rates. There are four levels of priority, including very high (0b11), high (0b10), medium (0b01), and low (0b00).
- **Hardware priority.** On STM32L4 processors, channel 1 has the highest hardware priority, and channel 7 has the lowest. Users cannot change the hardware priority of DMA channels. When multiple channels have the same software priority, the DMA engine uses their hardware priority to arbitrate the bus accesses.

19.4 Programming DMA

Each DMA channel has four registers: the *channel memory address register* (CMAR), the *channel peripheral address register* (CPAR), the *channel number of data register* (CNDTR), and the *channel configuration register* (CCR). As shown in Figure 19-7, CMAR and CPAR specify the starting address of the data memory and the peripheral, respectively. The transfer between the memory and the peripheral can take place in either direction.

The CCR register specifies the data transfer direction, whether increment mode is used, whether circular mode is used, and the channel priority. The CCR register is also used to enable or disable *transfer error interrupts*, *half-transfer interrupts*, and *transfer-complete interrupts*. Software must specify the total number of data to transfer in the CNDTR register.

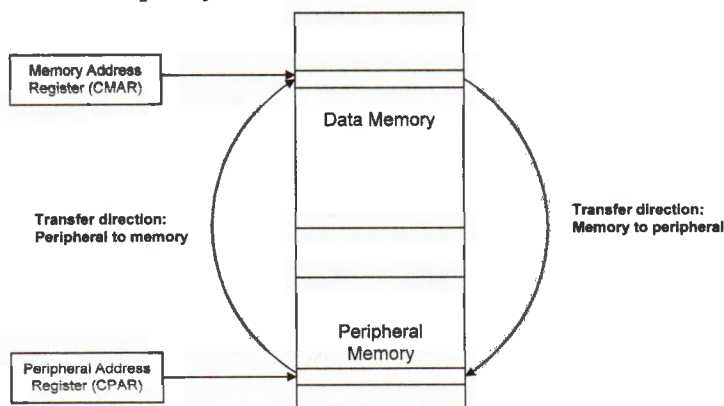


Figure 19-7. DMA transfer direction. On STM32L, memory-to-memory DMA is allowed, but peripheral-to-peripheral DMA is not.

Increment mode determines whether CPAR or CMAR increments after each DMA transfer, (see Figure 19-8). This mode can be programmed separately for CPAR and CMAR. For example, if increment mode for CMAR is enabled and *MSIZE* is four bytes, then the memory address stored in CMAR automatically increments by four after each DMA transfer.

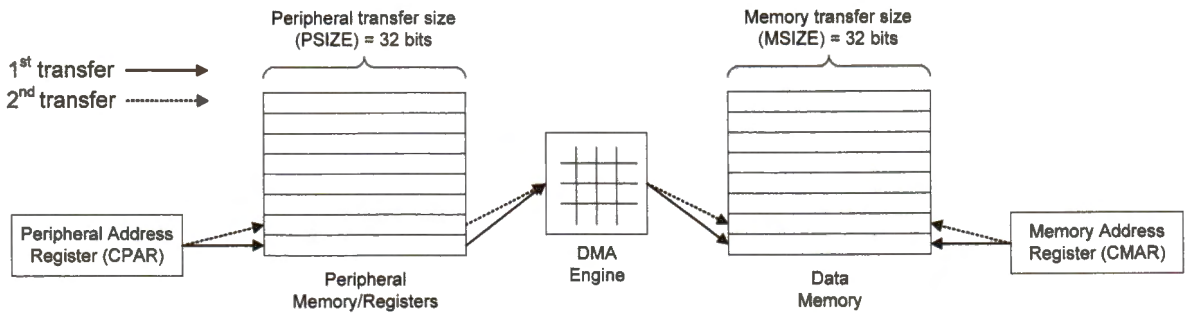


Figure 19-9 configures the *receive buffer not empty* (RXNE) event of USART 1 as the trigger of DMA 2 channel 7 by programming the channel selection register (CSELR). All available triggers that can make DMA requests on channel 7 of DMA 2 are listed in Table 19-2. When USART receives a byte, hardware sets the RXNE flag. If the channel enable bit (EN) is set in the CCR register, each RXNE event triggers one DMA transfer request. Example 19-1 gives the implementation in C.

```
RCC->AHB1ENR |= RCC_AHB1ENR_DMA2EN; // Enable DMA clock
DMA2_Channel7->CCR &= ~DMA_CCR_EN; // Disable DMA channel
DMA2_Channel7->CCR &= ~DMA_CCR_PSIZE; // Peripheral data size 00 = 8 bits
DMA2_Channel7->CCR &= ~DMA_CCR_MSIZE; // Memory data size: 00 = 8 bits
DMA2_Channel7->CCR &= ~DMA_CCR_PINC; // Disable peripheral increment mode
DMA2_Channel7->CCR |= DMA_CCR_MINC; // Enable memory increment mode
DMA2_Channel7->CCR &= ~DMA_CCR_DIR; // Transfer direction: to memory
DMA2_Channel7->CNDTR = 6; // Number of data to transfer
DMA2_Channel7->CPAR = (uint32_t)&(USART1->RDR); // Peripheral address
DMA2_Channel7->CMAR = (uint32_t) Buffer; // Receive buffer address
DMA2_CSELR->CSELR &= ~DMA_CSELR_C6S; // See Table 19-2
DMA2_CSELR->CSELR |= 2<<24; // Map channel 7 to USART1_RX
DMA2_Channel7->CCR |= DMA_CCR_EN; // Enable DMA channel 7
```

19.5 DMA Circular Mode

The DMA channel can run in *circular mode* or *normal mode*.

- If the CIRC bit in the CCR register is cleared, the DMA channel runs in *normal mode*. For each DMA transfer, hardware decrements the CNDTR register by 1. The DMA channel is disabled once CNDTR reaches 0. Once the DMA channel is enabled, software cannot modify CNDTR.
- If CIRC in CCR is set, the DMA channel runs in *circular mode*. Circular mode is often used for memory to implement a circular buffer. After CNDTR reaches 0, CPAR, CMAR, and CNDTR are automatically reset to the value originally programmed. Therefore, DMA can start the next round of DMA transfers. Circular mode is useful for DMA transfers for continuous data streams.

As shown in Figure 19-10, the CNDTR register of a DMA channel is set to 6. After the DMA completes 6 transfers, the buffer is automatically reused for subsequent DMA transfers.

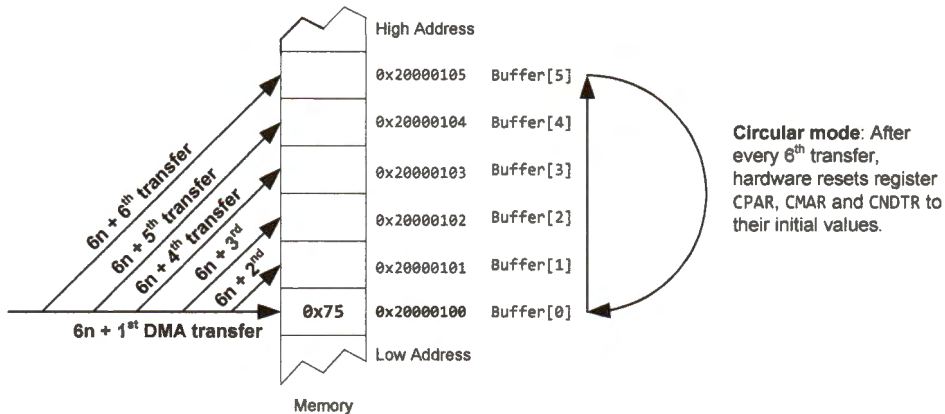


Figure 19-10. DMA in circular mode

Each DMA channel can generate three interrupts: transfer finished (TC), half-finished (HT), and transfer error (TE). The following code enables the TC interrupt of DMA 2 Channel 7.

```
// Enable the transfer complete interrupt
DMA2_Channel7->CCR |= DMA_CCR_TCIE;

// Disable the half transfer interrupt
DMA2_Channel7->CCR &= ~DMA_CCR_HTIE;

// Set the priority as the most urgent
NVIC_SetPriority(DMA2_Channel7_IRQn, 0);

// Enable NVIC interrupt
NVIC_EnableIRQ(DMA2_Channel7_IRQn);
```

Example 19-2. Enabling the TC interrupt for DMA 2 channel 7

19.6 DMA Interrupts

Software can identify the type of a DMA interrupt by checking the DMA interrupt status register (DMA_ISR), as shown in Example 19-3.

- Once half of the number of bytes (specified in the CNDTR register) are transferred, the half-transfer flag (HTIF) is set, and the DMA engine generates an interrupt if the half-transfer interrupt enable bit (HTIE) is set.
- At the end of the transfer, the transfer complete flag (TCIF) is set, and the DMA engine generates an interrupt if the transfer complete interrupt enable bit (TCIE) is set.
- If a DMA transfer accesses a restricted memory region, a DMA error takes place, and the hardware sets the transfer error flag (TEIF). The DMA controller automatically disables a faulty channel by clearing its EN bit, minimizing the performance impacts on the other active channels.
- If any of the transfer complete, half transfer and transfer error interrupts take place, hardware sets the global interrupt flag (GIF).
- Software clears these interrupt flags by writing 1 to the corresponding bit in the interrupt flag clear register (IFCR).

```
void DMA2_Channel17_IRQHandler(void) {
    if ( (DMA2->ISR & DMA_ISR_TCIF7) == DMA_ISR_TCIF7 ) {// transfer complete
        DMA2->IFCR |= DMA_IFCR_CTCIF7; // Clear flag TCIF by writing 1 to it
        ...
    }

    if ( (DMA2->ISR & DMA_ISR_HTIF7) == DMA_ISR_HTIF7 ) {// half transfer
        DMA2->IFCR |= DMA_IFCR_CHTIF7; // Clear flag IF7 by writing 1 to it
        ...
    }

    if ( (DMA2->ISR & DMA_ISR_GIF7) == DMA_ISR_GIF7 ) {// global interrupt
        DMA2->IFCR |= DMA_IFCR_CGIF7; // Clear flag HTIF by writing 1 to it
        ...
    }

    if ( (DMA2->ISR & DMA_ISR_TEIF7) == DMA_ISR_TEIF7 ) {// transfer error
        DMA2->IFCR |= DMA_IFCR_CTEIF7; // Clear flag CTEIF by writing 1 to it
        ...
    }
}
```

Example 19-3. Interrupt handler for DMA 2 channel 7

19.7 Exercises

1. How does the software know which specific event generates the interrupt when a DMA interrupt occurs?
2. Why is using DMA interrupt to retrieve data from a peripheral more efficient than repeatedly polling the peripheral?
3. Write an assembly program that uses DMA to copy 256 bytes of data from one memory region to another memory region.
4. Write an assembly program that uses DMA to perform analog-to-digital conversion (ADC). (Requires the background of Chapter 20).
5. Write an assembly program that uses DMA to perform digital-to-analog conversion (DAC). (Requires the background of Chapter 21)